TRƯỜNG ĐẠI HỌC PHENIKAA

KHOA HỆ THỐNG THÔNG TIN

TÀI LIỆU HỌC VÀ GIẢI THÍCH CODE

Mô hình hóa và kiểm chứng hình thức với mô hình Kripke

Hệ thống nghiên cứu: Smart Lock

CSE703093 - An toàn phần mềm

Giảng viên hướng dẫn: ThS. Vũ Quang Dũng

Lớp: [NHÓM ĐIỀN] | Học kỳ, năm học: [NHÓM ĐIỀN]

Thành viên	Họ và tên	MSSV
Người 1	[NHÓM ĐIỀN]	[NHÓM ĐIỀN]
Người 2	[NHÓM ĐIỀN]	[NHÓM ĐIỀN]
Người 3	[NHÓM ĐIỀN]	[NHÓM ĐIỀN]
Người 4	[NHÓM ĐIỀN]	[NHÓM ĐIỀN]

Tài liệu nội bộ phục vụ đọc code và chuẩn bị trả lời

Ngày tổng hợp bằng chứng: 15/10/2026

MỤC LỤC

1. Đọc chương trình theo một luồng chung 3

2. State và AP: dữ liệu nào thực sự được kiểm tra? 4

3. Guard/action trong model.py::step 5

4. BFS tạo states thật trong kripke.py 7

5. Dữ liệu Kripke, snapshot và trace 8

6. CTL trong code: đúng năm đặc tả 9

7. AF fixed-point và lasso P5 10

8. BMC: initial, domain và transition 11

9. Z3 nhận gì và trả gì? 12

10. Giải mã và replay counterexample 13

11. Tăng bound, completeness và phạm vi kết luận 14

12. Generator và chương trình Python độc lập 15

13. Đối chiếu mã sinh, sơ đồ và kết quả thật 16

14. Ôn tập, chạy lại và trách nhiệm nhóm 17

15. Câu hỏi giảng viên: mô hình và đặc tả 18

Mục lục được tạo tự động theo số trang PDF; số trang tính từ bìa. Các mục phụ lục nằm sau phần nội dung chính.

1. Đọc chương trình theo một luồng chung

Tài liệu này dành cho bốn thành viên học code đã có. Không thay thế báo cáo chính; ưu tiên trả lời "hàm nào làm gì, nhận gì, trả gì, vì sao đúng". Lý thuyết chỉ nhắc lại ở nơi cần đọc code. Mọi tên hàm và đoạn trích thuộc phiên bản G4 đã nghiệm thu, không phải pseudocode mới.

Luồng chính: model.py tạo State và bước nghiệp vụ → kripke.py dùng BFS sinh K → ctl.py kiểm năm thuộc tính → bmc.py mã hóa bốn safety → artifacts.py sinh Python, đối chiếu và vẽ → run.py chọn chế độ chạy.

1.1. Ví dụ xuyên suốt, phiên bản v1

```
s0 --password_ok--> s1 --open_door--> s5 --close_door--> s0

s0 --password_bad--> s2 --password_bad--> s6

s6 --password_bad--> s10 --tamper--> s14

s14 --admin_reset [True]--> s17 --reset_complete--> s0
```

Luồng thứ nhất xác thực/mở/đóng cửa. Luồng thứ hai khóa đúng lần sai thứ ba, phát báo động và phục hồi bằng quyền admin. Khi đọc mỗi hàm, hãy thử chỉ ra nó xử lý phần nào của hai luồng. ID phụ thuộc version: không lấy s5 v0 để suy ra giá trị s5 v1.

Thành phần	Input chính	Output chính
model.step	State, event, version, admin bool	State mới hoặc InvalidEvent
kripke.build_model	version	SmartLockModel với S/S0/R/L, cạnh, parent/depth
ctl.check_ctl	Cùng model G2	P1..P5: HOLDS/VIOLATED, trace nếu sai
bmc.check_safety_bmc	Cùng model, P1..P4, bound tùy chọn	Kết luận và từng query Z3 thật
artifacts.generate_python	model.json, path đầu ra	File Python độc lập

Bảng 1.1. Các hàm cần nhớ trước khi đọc chi tiết.

G2/G3/G4 có lần lượt 78/150/199 trường hợp pytest; G5 chạy lại 199 project tests và 4 tests Python G1, tổng 203 PASS, 0 FAIL. Không cộng 702 phép so bên trong audit vào số lượng tests pytest.

2. State và AP: dữ liệu nào thực sự được kiểm tra?

2.1. smartlock/model.py::State

Input constructor gồm sáu giá trị; output là object bất biến về trường. Mặc định tạo INITIAL_STATE. Các hàm replace trong action tạo State mới, không sửa khóa dictionary đã dùng bởi BFS. Equality/hash mặc định của dataclass so toàn bộ sáu trường, không so bằng ID hay tên mô tả.

smartlock/model.py : dòng 21-38

```
21  @dataclass(frozen=True)
22  class State:
23      authenticated: bool = False
24      failed_attempts: int = 0
25      door_open: bool = False
26      alarm: bool = False
27      phase: str = "normal"
28      admin_authorized: bool = False
29
30      def __post_init__(self) -> None:
31          """Chỉ ràng buộc miền; không loại bỏ trạng thái vi phạm nghiệp vụ."""
32          for name in ("authenticated", "door_open", "alarm", "admin_authorized"):
33              if type(getattr(self, name)) is not bool:
34                  raise ValueError(f"{name} phải là bool")
35          if type(self.failed_attempts) is not int or not 0 <= self.failed_attempts <= 3:
36              raise ValueError("failed_attempts phải là số nguyên từ 0 đến 3")
37          if self.phase not in PHASES:
38              raise ValueError(f"phase phải thuộc {PHASES}")
```

Bốn bool là authenticated, door_open, alarm, admin_authorized. Bộ đếm là int 0..3, phase là normal/locked_out/resetting. `type(x) is int` ngăn True lọt vào bộ đếm vì bool là lớp con int trong Python. Kiểm constructor là kiểm miền, không cấm state vi phạm P1. Nếu cấm cửa mở khi chưa xác thực tại đây, v0 không thể biểu diễn lỗi cần phát hiện.

2.2. smartlock/model.py::labels(state)

Input State; output frozenset AP đúng. Auth/door/alarm/admin đọc trực tiếp bool. Locked_out đọc phase, attempts_eq_3 đọc failed_attempts; resetting đọc phase; idle so cả State với INITIAL_STATE. Không phụ thuộc sN. Một State hợp lệ có tập nhãn rỗng là bình thường, ví dụ s2 v1 đang có f=1 nhưng chưa có AP nào trong tám tên được chọn.

Vì sao phải độc lập? Nếu locked_out được định nghĩa bằng f=3 thì P2 "locked_out kéo theo f=3" đúng do cách đặt tên. Nếu admin_authorized suy từ resetting thì P4 không kiểm được thiếu quyền. Hai lỗi đặc tả này bị tests kiểm bằng State hợp lệ về miền nhưng sai nghiệp vụ.

Tự kiểm: đọc s17 v1 từ model.json. Kết quả phải có admin_authorized, alarm, attempts_eq_3, resetting; không có idle, door_open hay authenticated. Đây là reset đang giữ alarm và bộ đếm, chưa hoàn tất.

3. Guard/action trong model.py::step

3.1. Authentication và Account Lockout

Input `step` là State hiện tại, event, version, input admin bool; output là State mới hoặc InvalidEvent. Hàm kiểm version và strict bool trước; ưu tiên nhánh resetting trước logic thông thường.

smartlock/model.py : dòng 89-100

```
 89  can_authenticate = (
 90      state.phase == "normal" and not state.alarm
 91      and not state.authenticated and not state.door_open
 92  )
 93  if event == "password_ok" and can_authenticate:
 94      return replace(state, authenticated=True, failed_attempts=0)
 95  if event == "password_bad" and can_authenticate and state.failed_attempts < 3:
 96      attempts = min(state.failed_attempts + 1, 3)
 97      return replace(
 98          state, authenticated=False, failed_attempts=attempts,
 99          phase="locked_out" if attempts == 3 else "normal",
100      )
```

`can_authenticate` yêu cầu normal, không alarm, chưa xác thực, cửa đóng. Password_ok trả replace với authenticated=True, f=0. Password_bad tính `attempts` mới, rồi dùng chính giá trị mới để chọn locked_out khi bằng 3. Nếu so giá trị cũ hoặc khóa khi f=2 sẽ sai yêu cầu. Đã lockout thì guard normal sai, không nhận cả password_ok.

Chạy tay v1: s0 có f=0; lần sai 1 → s2, f=1 normal; lần 2 → s6, f=2 normal; lần 3 → s10, f=3 locked_out. wait không xóa chuỗi sai. Sau hai lần sai, password_ok xóa f về 0; đóng cửa rồi thử sai lại phải bắt đầu từ 1. Unit test kiểm từng bước, không chỉ state cuối.

3.2. Door Control, Alarm

Open_door yêu cầu normal, không alarm, cửa đóng và đã xác thực ở v1. Trong code, `(state.authenticated or version == 'v0')` thể hiện chính BUG-01. V0 từ initial mở thẳng tới s3 chưa xác thực; v1 từ chối. Close_door đóng cửa và hủy phiên. Tamper đặt alarm=True, đóng cửa, hủy phiên trong cùng một replace, giữ f và phase.

Các biến không nêu trong replace giữ nguyên. State mới lại qua kiểm miền __post_init__. Hàm không lọc trạng thái bằng CTL và không thay đổi đầu vào của bên gọi. InvalidEvent có nghĩa không có bước chuyển được chấp nhận, không có nghĩa một self-loop tự động.

3.3. Admin Recovery và ưu tiên reset

smartlock/model.py : dòng 79-86

```
79
80  # F5: reset nội bộ có ưu tiên; không nhận input bên ngoài trong pha này.
81  if state.phase == "resetting":
82      if event == "reset_complete":
83          return INITIAL_STATE
84      if event == "reset_retry" and version == "v0":
85          return state  # BUG-02: có thể chọn thử lại vô hạn.
86      raise InvalidEvent(f"{event} bị từ chối khi đang resetting")
```

Đang resetting thì v1 chỉ trả INITIAL_STATE khi event complete. V0 cho reset_retry trả chính state. Mọi input khác bị từ chối ngay, nên không rơi xuống nhánh wait/tamper bên dưới. Thứ tự nhánh quyết định tiến triển: nếu để wait chạy trước sẽ vô tình thêm vòng lặp ở v1.

smartlock/model.py : dòng 115-123

```
115 # F5: lưu kết quả xác thực admin cho yêu cầu reset đã được chấp nhận.
116 if event == "admin_reset" and admin_authenticated:
117     return replace(
118         state, phase="resetting", admin_authorized=True,
119         authenticated=False, door_open=False,
120     )
121 if event == "wait":
122     return state  # Chỉ tới đây khi không ở resetting.
123 raise InvalidEvent(f"{event} không được phép ở trạng thái {state}")
```

Admin_reset cần input True tường minh. Action lưu quyền, đặt resetting, đóng cửa, hủy phiên; giữ f/alarm để thấy reset đang tiến hành. Complete mới xóa toàn bộ về initial. Password_ok của người dùng không thay input admin, không cho phép người dùng thường tự phục hồi.

Ví dụ v1: s14 là locked_out, f=3, alarm=True. Gọi admin_reset thiếu quyền phát sinh InvalidEvent. Truyền True thì tới s17 resetting, u=True, f=3, alarm=True. Complete trả s0; f/alarm/u đều được xóa. Input admin là giả định mô phỏng tin cậy, không phải mật khẩu thật hoặc state thứ bảy.

3.4. model.py::successors(state, version)

Hàm generator nhận State/version, yield các cặp event/State hợp lệ. Nó thử chín EVENTS theo thứ tự cố định bằng chính step. Riêng admin_reset truyền admin_authenticated=True để cạnh mang nghĩa đã xác thực quản trị. Chỉ bắt InvalidEvent để bỏ event bị từ chối; lỗi khác không được nuốt. Nhờ đó simulator và BFS không có hai bản guard/action khác nhau.

4. BFS tạo states thật trong kripke.py

4.1. build_model(version) và các cấu trúc dữ liệu

Input là version v0/v1. Output là SmartLockModel. Khởi tạo graph với s0/nhãn initial, `states={'s0': INITIAL_STATE}`, state_ids theo chiều ngược, distances[s0]=0, parents[s0]=None và deque chứa INITIAL_STATE. Không khởi tạo một danh sách 18/21 state hoặc đọc S từ JSON.

smartlock/kripke.py : dòng 164-179

```
164
165 while queue:
166     current = queue.popleft()
167     source = state_ids[current]
168     for event, next_state in successors(current, version):
169         if next_state not in state_ids:
170             target = f"s{len(states)}"
171             state_ids[next_state] = target
172             states[target] = next_state
173             graph.add_state(target, labels=labels(next_state))
174             distances[target] = distances[source] + 1
175             parents[target] = Transition(source, event, target)
176             queue.append(next_state)
177         target = state_ids[next_state]
178         transitions.append(Transition(source, event, target))
179         graph.add_transition(source, target)
```

Đọc từng dòng: popleft lấy state phát hiện sớm nhất chưa mở rộng; source lấy ID; successors sinh các đích bằng action. Nếu đích chưa biết, gán sN mới, thêm graph/nhãn, lưu depth bằng depth nguồn +1, lưu cạnh parent, append cuối queue. Sau nhánh if, vẫn ghi cạnh dù target cũ, để không làm mất self-loop hoặc đường quay về.

`state_ids` dùng State làm khóa, nên hai object có cùng sáu giá trị chỉ thành một node. FIFO và cạnh có chi phí một bước bảo đảm parent tạo đường ngắn nhất. Thứ tự EVENTS và insertion order ổn định giúp cùng version sinh cùng ID, JSON và trích đoạn minh họa.

4.2. Vì sao đúng 21/18 states?

V1 có 8 normal: ba state chưa xác thực f=0,1,2; hai state đã xác thực f=0 (cửa đóng/mở); ba state alarm f=0,1,2. Có hai locked_out với f=3, alarm False/True. Có tám resetting tương ứng f=0..3 × alarm False/True. Tổng 8+2+8=18. V0 thêm ba state mở cửa chưa xác thực f=0,1,2 → 21.

Đây là lời giải thích sau khi BFS và JSON xác nhận, không phải danh sách state đầu vào. 192 là tích miền; các tổ hợp như phase normal, f=3 không reachable. Không nên thêm state để đạt một số lượng định trước.

5. Dữ liệu Kripke, snapshot và trace

5.1. SmartLockModel và lớp toolkit

`states`: ID → State; `graph`: lớp KripkeStructure gốc giữ S/S0/R/L; `transitions`: list Transition(source,event,target); `parents` và `distances`: cây BFS. `_load_toolkit` dùng importlib nạp đúng file trong toolkit/softsec-toolkit/ch1_models/kripke.py; không sao chép/sửa toolkit.

`path_to(target)` theo parents ngược rồi đảo danh sách cạnh. Output là đường ngắn nhất để giải thích/replay state. `statistics()` đếm reachable, cạnh event, cặp R và max depth. `to_dict()` xuất đầy đủ trạng thái, AP, event assumptions, transition, R và kết quả kiểm tra; write_model ghi JSON.

11 structural_checks kiểm initial, reachable, total, labels, tính duy nhất, xác định theo event, khớp guard/action, R và cây BFS. Bốn basic invariants kiểm miền f, lockout/f3, alarm/cửa đóng, reset/quyền. Không dùng chúng lọc BFS nên không che BUG-01/BUG-02.

5.2. load_g2_model(path)

smartlock/kripke.py : dòng 193-203

```
193 def load_g2_model(path: Path) -> SmartLockModel:
194     """Dùng chính BFS G2 và yêu cầu khớp toàn bộ snapshot G2 trước G3.
195 
196     Không có bộ state/transition riêng dành cho bộ kiểm chứng.
197     Snapshot sai hoặc cũ phải báo lỗi thay vì bị ghi đè âm thầm.
198     """
199     snapshot = json.loads(path.read_text(encoding="utf-8"))
200     model = build_model(snapshot["version"])
201     if snapshot != model.to_dict():
202         raise ValueError(f"Snapshot G2 không khớp guard/action/BFS hiện tại: {path}")
203     return model
```

Input là JSON snapshot, output object G2. Hàm xây lại bằng cùng BFS, so toàn bộ dictionary chứ không chỉ số state. Snapshot lỗi/cũ phát sinh ValueError; không âm thầm ghi đè. CTL/BMC/generator dùng kết quả này.

`make_trace(model,state_ids,events,pid)` ghép n+1 ID/n event với đủ sáu giá trị/labels; incoming_event và bool quyền lưu ở row đích. Nó không tự tìm đường. `replay_trace(model,trace)` kiểm initial, từng row, cạnh và action. Output True nếu hợp lệ; exception nếu có bất đồng.

R total chỉ có nghĩa mỗi node có ít nhất một kế tiếp, không phải mọi event đều hợp lệ. 69/46 event edge lớn hơn 65/42 cặp R vì tamper và wait tại bốn state alarm ngoài reset cùng tự lặp; R gộp cặp, list cạnh giữ tên.

6. CTL trong code: đúng năm đặc tả

6.1. FORMULAS và bad_state(state, property_id)

FORMULAS lưu chuỗi để trình bày, không phải đầu vào parser. Labels đọc State. P1-P4 phủ định mệnh đề trạng thái trong AG để tìm bad; P5 tách riêng vì cần phân tích mọi đường tương lai.

P	Công thức chính xác	Cách quyết định
P1	AG(door_open -> (authenticated AND NOT locked_out))	Không reachable bad1
P2	AG(locked_out -> attempts_eq_3)	Không reachable bad2
P3	AG(alarm -> NOT door_open)	Không reachable bad3
P4	AG(resetting -> admin_authorized)	Không reachable bad4
P5	AG(resetting -> AF idle)	Mọi resetting thuộc AF idle

Bảng 6.1. Safety P1-P4 và liveness P5.

smartlock/ctl.py : dòng 30-41

```
30  def bad_state(state: State, property_id: str) -> bool:
31      """Phủ định mệnh đề trạng thái của riêng P1–P4, đọc AP từ State."""
32      ap = labels(state)
33      bad = {
34          "P1": "door_open" in ap and ("authenticated" not in ap or "locked_out" in ap),
35          "P2": "locked_out" in ap and "attempts_eq_3" not in ap,
36          "P3": "alarm" in ap and "door_open" in ap,
37          "P4": "resetting" in ap and "admin_authorized" not in ap,
38      }
39      if property_id not in bad:
40          raise ValueError("bad_state chỉ áp dụng P1–P4, không áp dụng P5")
41      return bad[property_id]
```

Input bad_state là State/P1..P4, output bool. Nó không nhận ID để đoán nhãn. P5 bị từ chối thay vì coi resetting là bad. `check_ctl` quét mọi reachable state, chọn bad có depth nhỏ nhất, dùng path_to/make_trace, đánh dấu violation_step rồi validate bằng chứng.

`validate_model` kiểm cấu trúc và tính lại BFS bằng reachable_depths, không chỉ tin distances đã lưu. Một graph thiếu cạnh hoặc không total không được dùng để kết luận toàn cục. `antecedent_states` trong kết quả ghi tiền đề có xuất hiện thật; ví dụ tám resetting của mỗi version khiến P4/P5 không được coi là thỏa do tiền đề rỗng.

7. AF fixed-point và lasso P5

7.1. ctl.py::af_idle_fixed_point(model)

Input graph total; output `(reached, history)`: tập state thỏa AF idle và từng vòng để giải thích. Bắt đầu từ idle, không bắt đầu toàn S. Một state chỉ thêm khi tập kế tiếp là tập con reached cũ.

smartlock/ctl.py : dòng 95-109

```
 95 def af_idle_fixed_point(model: SmartLockModel) -> tuple[set[str], list[list[str]]]:
 96     """μZ.(idle ∪ AX Z), trên quan hệ total; trả tập AF idle và các vòng lặp."""
 97     if any(not model.graph.transitions.get(sid) for sid in model.states):
 98         raise ValueError("AF yêu cầu Kripke total, không chấp nhận deadlock")
 99     order = list(model.states)
100     reached = {sid for sid, state in model.states.items() if "idle" in labels(state)}
101     history = [[sid for sid in order if sid in reached]]
102     while True:
103         expanded = reached | {
104             sid for sid in order if model.graph.transitions[sid] <= reached
105         }
106         if expanded == reached:
107             return reached, history
108         reached = expanded
109         history.append([sid for sid in order if sid in reached])
```

`expanded = reached | {...}` là phép mở rộng đơn điệu. Khi không đổi, trả điểm bất động nhỏ nhất. Dấu <= ở set là quan hệ tập con, không là so sánh ID số. Nếu dùng giao khác rỗng sẽ kiểm "tồn tại kế tiếp" và có thể chấp nhận sai v0. Check deadlock tránh điều kiện tập rỗng đúng rỗng.

V0 history=[{s0}], tám resetting ngoài tập. V1 history có kích thước [1,9], gồm idle và tám resetting. Normal có wait có thể không thuộc AF idle; P5 vẫn thỏa vì chỉ yêu cầu AF tại resetting.

7.2. ctl.py::_liveness_trace và validate_counterexample

Input là một resetting ngoài AF. Output lasso đã validate. Hàm lấy prefix BFS, theo cạnh có target ngoài AF; seen ánh xạ ID sang vị trí trong trace. Lặp ID đóng chu trình; vì graph hữu hạn, quá trình tìm sẽ gặp lại ID. V0 trả s0,s5,s5, reset_step=loop_start=1.

Validator đầu tiên gọi replay_trace, sau đó kiểm kind=lasso, chỉ số đúng, chu trình không rỗng/khép kín và không có idle từ reset_step. Không chấp nhận lasso reset_complete về s0 rồi reset lại, vì reset ban đầu đã hoàn tất. Kết quả liveness không đến từ Z3 bad-state.

8. BMC: initial, domain và transition

8.1. bmc.py::transition_constraint(model, source, event, target)

Input là model và ba biểu thức Z3; output một Or các And cho từng transition G2. ID state là vị trí trong dictionary model.states, event ID là vị trí trong EVENTS. Cả hai ánh xạ được dùng lại khi giải mã.

smartlock/bmc.py : dòng 10-17

```
10  def transition_constraint(model: SmartLockModel, source, event, target):
11      """T(x,e,y): OR đúng các cạnh source/event/target đã sinh ở G2."""
12      ids = {sid: i for i, sid in enumerate(model.states)}
13      event_ids = {name: i for i, name in enumerate(EVENTS)}
14      return Or([
15          And(source == ids[edge.source], event == event_ids[edge.event], target == ids[edge.target])
16          for edge in model.transitions
17      ])
```

Ví dụ v0 có nhánh source=0 AND event=2 AND target=3. Nếu bỏ event, solver sẽ không chỉ ra được thao tác nào thực hiện cạnh. Tests liệt kê mọi bộ ba thỏa T bằng solver và so đúng toàn bộ list G2, bắt cả cạnh thừa/thiếu. Không khai báo một hệ trạng thái khác trong Z3.

8.2. bmc.py::encode_path(model, bound)

Input k là int không âm, không chấp nhận bool. Output ba list: states gồm k+1 Int; events gồm k Int; constraints gồm initial/domain/T. Hàm chưa tạo Solver và chưa thêm bad.

smartlock/bmc.py : dòng 20-32

```
20  def encode_path(model: SmartLockModel, bound: int) -> tuple[list, list, list]:
21      """Unroll k bước: k+1 biến state ID, k biến event, I/domain/T."""
22      if type(bound) is not int or bound < 0:
23          raise ValueError("bound phải là số nguyên không âm")
24      states = [Int(f"x_{i}") for i in range(bound + 1)]
25      events = [Int(f"e_{i}") for i in range(bound)]
26      ids = {sid: i for i, sid in enumerate(model.states)}
27      constraints = [Or([states[0] == ids[sid] for sid in model.graph.initial])]
28      constraints += [And(state >= 0, state < len(ids)) for state in states]
29      for i, event in enumerate(events):
30          constraints.append(And(event >= 0, event < len(EVENTS)))
31          constraints.append(transition_constraint(model, states[i], event, states[i + 1]))
32      return states, events, constraints
```

Với k=1 có x0,e0,x1. Initial yêu cầu x0 bằng ID initial; domain chặn giá trị ngoài S/E; T chặn nhảy qua cạnh không có trong graph. Với k=0 vẫn có x0/initial/domain để phát hiện nếu lỗi ngay trạng thái đầu. K là số bước, không phải số state. BMC_j(k) thêm OR bad_j ở mọi xi, vì lỗi có thể xuất hiện trước bước cuối rồi được phục hồi sau đó.

9. Z3 nhận gì và trả gì?

9.1. bmc.py::check_bound(model, property_id, bound)

Input model/P1-P4/k; output dictionary của một query thật. Hàm validate model, encode_path, tính bad_ids từ State. Tạo Solver, timeout, gửi initial/domain/T và điều kiện bad; lưu solver.to_smt2 rồi gọi check.

smartlock/bmc.py : dòng 51-63

```
51  ids = list(model.states)
52  bad_ids = [i for i, sid in enumerate(ids) if bad_state(model.states[sid], property_id)]
53  solver = Solver()
54  solver.set(timeout=10000)
55  solver.add(*constraints)
56  bad = Or([state == bad_id for state in states for bad_id in bad_ids]) if bad_ids else BoolVal(False)
57  solver.add(bad)  # Tập bad rỗng vẫn gọi solver, SMT-LIB ghi rõ (assert false).
58  result["smt2"] = solver.to_smt2()
59  result["solver_called"] = True
60  outcome = solver.check()
61  result["solver_status"] = str(outcome).upper()
62  if outcome == sat:
63      witness = solver.model()
```

`solver.add(*constraints)` thêm cả danh sách đường đi. `solver.add(bad)` thêm mục tiêu tồn tại đường vi phạm, không thêm thuộc tính an toàn như giả định. Bad rỗng được mã hóa False nhưng vẫn gọi check để có UNSAT thật. Nếu graph đã được liệt kê đầy đủ và bad rỗng, điều này cũng cho thấy BMC không thay thế công việc sinh mô hình/đặc tả.

SAT: có model thỏa công thức, chỉ khi đó gọi solver.model. UNSAT: không có witness cho query với k hiện tại. UNKNOWN: chưa quyết định, lưu reason và không kết luận thỏa. ERROR là exception ứng dụng, không phải phản hồi solver. Ba trường solver_called/solver_status/status được tách để không biến nhánh chưa chạy hoặc replay sai thành UNSAT.

9.2. Đọc query thực tế

P1 v0 k=0 UNSAT; k=1 SAT, witness (x0,e0,x1)=(0,2,3). P1 v1 không có bad nên k=0..5 đều UNSAT. P2-P4 đều UNSAT trên cả hai. Hai JSON G3 lưu 44 query: 1 SAT, 43 UNSAT, 0 UNKNOWN; G4/G5 không thay số liệu này bằng số lần solver chạy trong tests.

Header SMT2 `:status unknown` do hàm xuất tạo không phải kết quả check. Luôn đọc solver_status. Tests UNKNOWN/ERROR dùng lớp giả lập, được ghi rõ; chưa có UNKNOWN tự nhiên trong lần kiểm chứng mô hình nhỏ này.

10. Giải mã và replay counterexample

10.1. Giải mã trong bmc.py::check_bound

Sau SAT, witness.eval đọc từng Int bằng model_completion. as_long đổi về int; kiểm phạm vi trước khi tra ID/event. make_trace thêm dữ liệu State thật, tìm bước bad đầu tiên, validate rồi mới công bố SAT hợp lệ.

smartlock/bmc.py : dòng 64-77

```
64  result["solver_model"] = str(witness)
65  state_numbers = [witness.eval(state, model_completion=True).as_long() for state in states]
66  event_numbers = [witness.eval(event, model_completion=True).as_long() for event in events]
67  if (any(not 0 <= i < len(ids) for i in state_numbers)
68          or any(not 0 <= i < len(EVENTS) for i in event_numbers)):
69      raise ValueError("Model Z3 chứa ID ngoài miền")
70  trace = make_trace(model, [ids[i] for i in state_numbers],
71                     [EVENTS[i] for i in event_numbers], property_id)
72  trace["violation_step"] = next(
73      i for i, row in enumerate(trace["steps"])
74      if bad_state(model.states[row["state_id"]], property_id)
75  )
76  validate_counterexample(model, trace, property_id)
77  result.update(status="SAT", counterexample=trace, replay_validated=True)
```

Input n+1 state ID/n event cho make_trace phải khớp số lượng. Với P1 v0, output hai row: s0 idle và s3 door_open nhưng chưa authenticated; incoming_event=open_door, violation_step=1. Trạng thái vẫn là đủ sáu giá trị, không chỉ số 3 do solver cung cấp.

10.2. kripke.py::replay_trace(model, trace)

Hàm kiểm row đầu thuộc initial, không có event đầu vào, giá trị bằng INITIAL_STATE. Với mỗi row tiếp theo: chỉ số bước, dữ liệu State và labels phải đúng; admin input là bool và đúng nghĩa event; cạnh Transition phải có trong G2; gọi step thực tế phải ra State đích.

Replay chỉ chứng minh đường hợp lệ. `ctl.py::validate_counterexample` còn kiểm điều kiện thuộc tính: safety có bad và đúng violation_step; P5 có lasso khép kín tránh idle sau reset. Tách hai mức này giúp không gọi một đường bình thường là bằng chứng vi phạm.

Nếu solver SAT nhưng replay exception, query giữ solver_status=SAT, status=ERROR; không công bố trace là hợp lệ. Tests cấy sai cạnh, nhãn, input admin hoặc lasso qua idle để kiểm việc phát hiện sai lệch.

11. Tăng bound, completeness và phạm vi kết luận

11.1. bmc.py::check_safety_bmc(model, pid, max_bound)

Input P1-P4, bound tùy chọn; output kết luận cùng list checks. Hàm tính threshold bằng max reachable_depths, giới hạn limit bởi D, gọi check_bound từ 0 tăng dần. SAT thì dừng/VIOLATED; UNKNOWN thì dừng/ INCONCLUSIVE; ERROR thì dừng/ERROR. Không bỏ qua bound chưa quyết định.

smartlock/bmc.py : dòng 104-116

```
104  status = "BOUNDED_NO_COUNTEREXAMPLE"
105  for bound in range(limit + 1):
106      check = check_bound(model, property_id, bound)
107      checks.append(check)
108      if check["status"] == "SAT":
109          status, holds = "VIOLATED", False
110          break
111      if check["status"] in ("UNKNOWN", "ERROR"):
112          status = "INCONCLUSIVE" if check["status"] == "UNKNOWN" else "ERROR"
113          break
114  else:
115      if limit == threshold:
116          status, holds = "HOLDS", True
```

Nếu kết thúc vòng for bình thường, chỉ khi limit==threshold mới đặt holds=True/HOLDS. Với limit<D, holds vẫn None và status vẫn BOUNDED_NO_COUNTEREXAMPLE. Dừng ở SAT là phản ví dụ ngắn nhất khi mọi bound trước thực sự UNSAT; không chỉ vì lấy một đường BFS ngắn.

11.2. Vì sao D=5 đủ cho bốn safety?

D là max khoảng cách ngắn nhất từ s0 tới mọi reachable state. Đường ba lần sai/tamper/admin_reset dài 5 chứng minh có state ở mức này; BFS độc lập kiểm không có state nào cần đường ngắn nhất dài hơn. Nếu có bad, có đường tới nó không quá D. Totality cho phép kéo dài tiền tố tới đúng D, còn query kiểm bad tại mọi vị trí, nên UNSAT ở D đủ loại bad-state reachable của nhóm hiện tại.

D không là ngưỡng tối thiểu riêng từng công thức và không là |S|. Không dùng nó để loại chu trình P5; D=5 vẫn cho phép s5 v0 tự lặp vô hạn. Ngoài mô hình hữu hạn, các chứng minh về timing, hỏng phần cứng hoặc cơ chế mật khẩu thật đều chưa có trong đề tài.

12. Generator và chương trình Python độc lập

12.1. artifacts.py::generate_python(model_path, destination)

Input snapshot/path, output file Python. Hàm đọc states/initial/labels/ event assumptions/transitions. STATE_FIELDS lấy đúng sáu trường G2; EVENTS giữ đúng chín tên. Kiểm trùng ID, initial và miền; sau đó tạo bảng khóa (source,event). Một khóa có hai target khác nhau gây lỗi trước khi ghi file, không chọn ngẫu nhiên.

smartlock/artifacts.py : dòng 88-96

```
88  table = {}
89  for edge in data["transitions"]:
90      pair = edge["source"], edge["event"]
91      target = edge["target"]
92      if pair[0] not in states or target not in states or pair[1] not in EVENTS:
93          raise ValueError(f"Cạnh có state/event không hợp lệ: {edge}")
94      if pair in table and table[pair] != target:
95          raise ValueError(f"Nondeterministic transition {pair}: {table[pair]} / {target}")
96      table[pair] = target
```

Các bảng nhúng STATES, LABELS và TRANSITIONS là literal repr của dữ liệu JSON. VERSION và SHA-256 truy nguồn; EVENT_INPUTS bảo toàn điều kiện admin. RUNTIME chỉ là bộ tra bảng/CLI, không có nhánh tự tính f, quyền mở cửa hoặc báo động. Sinh lại cùng đầu vào cho cùng bytes.

12.2. generated/v1/smart_lock.py::step

generated/v1/smart_lock.py : dòng 119-132

```
119 def step(state, event, *, admin_authenticated=False):
120     if type(admin_authenticated) is not bool:
121         raise ValueError("admin_authenticated must be bool")
122     state_values(state)
123     if type(event) is not str:
124         raise InvalidEvent(f"Unknown event: {event!r}")
125     required = EVENT_INPUTS.get(event, {})
126     if required.get("admin_authenticated", False) and not admin_authenticated:
127         raise InvalidEvent("Admin verification is required")
128     try:
129         return TRANSITIONS[(state, event)]
130     except KeyError:
131         raise InvalidEvent(f"Event {event!r} is not enabled at {state}") from None
```

Hàm nhận ID/event/admin bool, kiểm State tồn tại và điều kiện quyền, tra bảng để trả ID mới. state_values trả dict sáu giá trị. CLI main in từng bước và trả 1 khi InvalidEvent. File chỉ import argparse/json, không cần toolkit/model.py hoặc JSON bên cạnh khi chạy.

V0 giữ lỗi từ bảng; v1 thiếu (s0,open_door), resetting chỉ có complete. Từ chối input không là cạnh. Caller gửi complete; runtime không có scheduler.

13. Đối chiếu mã sinh, sơ đồ và kết quả thật

13.1. artifacts.py::audit_generated(model, program)

Input model G2/file sinh; output số đo và stdout hoặc exception. Nạp bằng runpy với tên không phải __main__, so initial, tập State/event, toàn bộ giá trị/labels. So bảng cạnh nhúng **bằng** tập G2, không chỉ kiểm vài input; nhờ vậy bắt cả event backdoor nằm ngoài chín tên.

Sau đó lặp mọi State × EVENTS × {False,True}. Gọi step gốc lấy kỳ vọng, step sinh lấy kết quả; so đích hoặc từ chối. Chỉ bắt InvalidEvent để không che exception lập trình. Thu các bộ ba thực hiện và yêu cầu phủ hết đúng 69/46 cạnh, không thiếu và không thừa.

Phiên bản	Phép so	Chấp nhận	Từ chối	Cạnh khớp
v0	378	125	253	69
v1	324	82	242	46

Bảng 13.1. Số đo từ JSON, không ghi cứng trong phép audit. 702 là số bộ input hữu hạn; 115 là số transition phân biệt, không phải cùng một đại lượng. Chấp nhận lặp cùng cạnh với hai giá trị quyền vẫn tính hai input.

Chạy subprocess -I -S kiểm độc lập. Test còn copy duy nhất file sang thư mục trống. Tests chức năng, replay mọi đường BFS, cả hai CE v0 và mọi resetting v1 bổ sung cho đối chiếu đầy đủ một bước. Initial và quan hệ trùng cho phép quy nạp các chuỗi bước hợp lệ trùng nhau.

13.2. Các hàm tạo hình

`diagram_views(models,reports)` chọn hai full graph, ba lát cắt và hai trace G3. `_part` giữ ID/cạnh thật. `dot_text` tạo DOT, gộp event chung đầu/cuối, đánh dấu initial và đỏ P1/P5; P5 prefix xanh/cycle đỏ lấy từ loop_start. `resolve_dot` tìm executable; `render_view` gọi Graphviz thật, parse SVG/header PNG, lưu cảnh báo và kích thước. Lát cắt không thay graph kiểm chứng; build_artifacts đọc G2/G3, không gọi lại BMC.

14. Ôn tập, chạy lại và trách nhiệm nhóm

14.1. Trình tự đọc để không học thuộc kết quả

Mỗi người cần chạy một luồng thành công, một input bị từ chối và hai phản ví dụ v0; giải thích sáu giá trị ở từng bước. Sau đó đọc file mình phụ trách và ít nhất một file liền trước/sau trong luồng dữ liệu. Phân công cụ thể ở docs/PhanCongNhom.md; tên/MSSV còn chờ nhóm điền.

P	Phương pháp	v0	v1	Bound safety
P1	Đồ thị + Z3	VIOLATED	HOLDS	v0: 0..1
P2	Đồ thị + Z3	HOLDS	HOLDS	0..5
P3	Đồ thị + Z3	HOLDS	HOLDS	0..5
P4	Đồ thị + Z3	HOLDS	HOLDS	0..5
P5	CTL fixed-point	VIOLATED	HOLDS	Không áp dụng

Bảng 14.1. P1 v1 có bound 0..5, không dừng ở 1 như v0. P5 không gọi solver bad-state. Với v0, tests PASS xác nhận lỗi cố ý, không tuyên bố an toàn. G5 không sửa mã được nghiệm thu G4.

```
python -m pip install -r requirements.txt
python -m pytest --color=no -p no:cacheprovider -v
python -I -S generated/v0/smart_lock.py --events open_door
python -I -S generated/v1/smart_lock.py --events open_door
```

Lệnh v1 mở ngay trả 1 đúng mong đợi; thêm password_ok trước sẽ mở được. Chạy G2/G3/G4 vào tmp/recheck theo README để giữ bằng chứng cũ. Đọc query trong verification.json để thấy constraints/smt2/check thật, không suy SAT từ công thức hoặc từ tên bản buggy.

Nguồn biên soạn: docs/G1_ThietKe.md, G2_GiaiThich.md, G3_GiaiThich.md, G4_GiaiThich.md và mã/JSON/log G1-G4. Tài liệu chính thức liệt kê nguồn giảng viên, toolkit, Z3, NuSMV và Graphviz ở Chương 7. Codex/ChatGPT có hỗ trợ đáng kể trong mã G1-G4 và tài liệu G5; nhóm cần xác minh/hiểu và chịu trách nhiệm, không tuyên bố tự viết toàn bộ mà không có AI.

15. Câu hỏi giảng viên: mô hình và đặc tả

Câu 1. Vì sao dùng Kripke cho Smart Lock?

State gom sáu biến liên quan, R biểu diễn mọi bước hợp lệ và L cung cấp AP cho CTL. Cùng graph kiểm tra được quyền mở cửa lẫn mọi nhánh reset; năm chức năng không bị chia thành năm graph mất tương tác.

Câu 2. Vì sao BFS thay vì viết states thủ công?

build_model chỉ bắt đầu initial và gọi successors/step. Điều này sinh hết reachable theo guard/action, tránh bỏ state kết hợp alarm/lockout. Parent/depth của lần khám phá đầu cho đường ngắn nhất và D chính xác.

Câu 3. Vì sao 21/18, không phải 192 states?

192 là tích miền. V1 reachable có 8 normal +2 locked_out +8 resetting. V0 thêm ba cửa mở chưa xác thực với f=0,1,2. JSON/BFS xác nhận các tập này; code không nhập trước số 18 hoặc 21 để xây S.

Câu 4. Vì sao event transition nhiều hơn cặp R?

R là set cặp source/target. Ở bốn state alarm ngoài reset, tamper và wait cùng tự lặp nên R gộp chúng. List Transition giữ event để giải mã, mô phỏng và gắn nhãn; kết quả 69/65 và 46/42 không mâu thuẫn.

Câu 5. admin_authenticated có làm tăng lên bảy biến không?

Không. Đây là input của lần gọi step. State vẫn sáu trường; kết quả quyền lưu trong admin_authorized. Successors tạo cạnh admin_reset với input True; runtime sinh đọc EVENT_INPUTS để không cho caller thiếu quyền.

Câu 6. Tại sao dùng CTL thay vì chỉ assert trong unit test?

CTL AG/AF diễn tả mọi đường. Unit test chọn các input cụ thể, còn check_ctl xét toàn graph; AF phân biệt "có thể hoàn tất" với "mọi nhánh đều hoàn tất". Test vẫn cần để kiểm đặc tả/logic và chính checker.

Câu 7. Z3 thực sự kiểm chứng điều gì?

check_bound gửi I/domain/T và OR bad trên k+1 vị trí. Z3 quyết định có phép gán ID/event thỏa đường vi phạm không. Nó không đọc tiếng Việt, không tự suy CTL, không kiểm phần cứng và không trực tiếp kiểm P5.

Câu 8. Vì sao vừa CTL explicit vừa BMC/Z3?

Explicit quét S cho safety và fixed-point cho liveness. BMC thể hiện mã hóa SMT theo bound, cho witness ngắn nhất khi tăng k. Cả hai dùng G2 và được đối chiếu, đáp ứng hướng Kripke+BMC của giảng viên.

Câu 9. Khi nào SAT, UNSAT, UNKNOWN?

SAT khi công thức đường vi phạm có nghiệm; UNSAT khi không có nghiệm trong bound. UNKNOWN là chưa quyết định, không được đổi thành UNSAT. P1 v0 k=1 SAT, k=0 UNSAT; query thật không có UNKNOWN, tests giả lập nhánh đó.

Câu 10. Nếu tập bad rỗng, có thật gọi Z3 không?

Có. check_bound thêm BoolVal(False), vẫn Solver.check, lưu smt2 và solver_called=True. Không suy ra nhãn UNSAT mà bỏ lời gọi. Đồng thời phải hiểu bad rỗng đến từ graph BFS đầy đủ, không do ép thuộc tính vào T.

Câu 11. Vì sao P5 dùng fixed-point?

Không có một resetting đơn lẻ tự chứng minh reset không bao giờ xong. af_idle_fixed_point thêm state khi mọi kế tiếp đã chắc tới idle; complement cho nhánh tránh idle và lasso. Safety bad-state không đủ.

Câu 12. Lasso v0 chính xác là gì?

Prefix s0 --admin_reset [True]--> s5. Cycle s5 --reset_retry--> s5, lặp vô hạn. reset_step=loop_start=1; sau điểm reset không có idle. Có reset_complete tồn tại không cứu AF vì AF đòi mọi đường.

Câu 13. D=5 là gì, có phải kiểm mọi đường dài 5 không?

Là max khoảng cách ngắn nhất tới mọi reachable state, tính bằng BFS trên R. Một bad reachable có đường tới không quá 5. BMC mã hóa mọi đường k bước bằng solver, không cần Python liệt kê từng đường riêng.

Câu 14. UNSAT ở k=0 có chứng minh v0 an toàn không?

Không. Initial không bad nhưng open_door ở bước 1 gây P1 sai. Nếu giới hạn k=0, check_safety_bmc trả BOUNDED_NO_COUNTEREXAMPLE/holds=None. Chỉ khi đủ ngưỡng của safety mới kết luận HOLDS toàn graph.

Câu 15. Vì sao P2 không thay test lần sai thứ ba?

P2 chỉ nói lockout kéo theo f=3, không bảo đảm ngay f mới bằng 3 thì pha đổi đúng lúc. Test gọi password_bad ba lần, kiểm lần 1/2 normal và lần 3 locked_out; resetting giữ f=3 không trái P2.

Câu 16. Replay khác solver SAT như thế nào?

SAT chỉ chứng minh encoding có nghiệm. replay_trace kiểm nghiệm đó đúng initial/cạnh/action/nhãn/quyền G2; validate_counterexample kiểm thêm vi phạm. Encoding hoặc giải mã sai phải bị phát hiện, không che bằng SAT.

Câu 17. Có thể bật admin bằng password_ok không?

Không. Password_ok chỉ đặt authenticated và xóa f. Admin_reset còn cần input bool True riêng. Tests trên State đã đăng nhập nhưng input False vẫn phải bị từ chối; labels quyền đọc trường riêng.

Câu 18. Vì sao v0 lỗi mà tests vẫn PASS?

Các test v0 mong phát hiện mở cửa không quyền và self-loop retry. PASS nghĩa mã tái hiện đúng thiết kế thử nghiệm. Kiểm chứng trả VIOLATED cho P1/P5, không gọi v0 an toàn từ số test đạt.

Câu 19. Hai sửa đổi v1 loại lỗi bằng cách nào?

Guard mở cửa cần authenticated, và action hủy phiên đồng thời đóng cửa nên loại reachable bad1. Resetting v1 chỉ có complete về idle, mọi kế tiếp nằm trong Z0 nên tám resetting được thêm vào AF ngay vòng đầu.

Câu 20. Vì sao không thêm wait vào mọi state cho "total"?

Total cần một kế tiếp, không cần self-loop. V1 resetting đã có complete; thêm wait tạo đường ở mãi resetting và làm P5 sai. Ngoài reset wait được cho phép rõ ràng; các input từ chối không tạo cạnh chờ.

Câu 21. Mã sinh có thật tương ứng với mô hình không?

Audit so toàn bộ States/labels/initial và bảng cạnh; 702 input bao phủ mọi reachable state, chín event, hai quyền. 115/115 cạnh khớp, không có thêm/bớt; test còn chạy file một mình với -I -S và replay mọi đường BFS.

Câu 22. Một state/event có hai đích thì generator làm gì?

generate_python phát sinh ValueError Nondeterministic transition trước khi ghi file. Test thêm target khác cho một khóa thật rồi kiểm không có file đầu ra. Không chọn đích đầu tiên hoặc chọn ngẫu nhiên.

Câu 23. Có thể dùng lát cắt sơ đồ để kiểm lại CTL không?

Không. Bỏ cạnh để dễ nhìn có thể loại một nhánh xấu và đổi nghĩa A. diagram_views chỉ để giải thích; CTL/BMC dùng toàn bộ model. Tests xác nhận mọi node/cạnh của hình đều trích thật từ graph tích hợp.

Câu 24. Nếu caller không gửi complete thì P5 v1 còn đúng không?

P5 vẫn là kết quả trên K đã định nghĩa, nơi resetting chỉ có cạnh complete. Runtime không có scheduler/thời gian thực; caller dừng ngoài mô hình không được tính thành bước wait hợp lệ. Không suy bảo đảm theo giây.

Câu 25. Nhóm phải tự làm gì trước khi nộp?

Điền bìa, tên/MSSV/lớp/học kỳ và phân công thực tế; đọc code, chạy lại lệnh README, giải thích hai CE cùng giới hạn. Giữ khai báo AI đúng mức hỗ trợ; đổi tên gói theo lớp. Không sửa số liệu để đẹp hoặc gọi an toàn tuyệt đối.